

人工智能基础 A 综合实践实验报告

一、基本信息

是否组队		☒ 是 ☐ 否	组名	碱基互补配队
成员信息	成员 1	姓名： <u>蒋贤迪</u> 学号： <u>3240105782</u>		
	成员 2	姓名：_____ 学号：_____		
	成员 3	姓名：_____ 学号：_____		
选择框架		☒ PyTorch ☐ TensorFlow ☐ scikit-learn ☐ 飞桨 ☐ 其他：_____		
选题		基于 RNA-FM 与 RNA-ret 的 RNA 二级结构预测平台 (RNA-Dee)		
数据集		☐ 中药数据集 ☒ 其他： <u>RNAStrAlign</u>		
基准测试集		ArchiveII		

数据集说明：

本实验采用 RNAStrAlign 和 ArchiveII 数据集。该数据涵盖了 tRNA, 5S rRNA, 16S rRNA 等多个 RNA 家族。

- 数据格式：原始数据为 .bpseq 格式（包含 Index, Residue, Pair Index 三列），经预处理脚本解析为序列与 2D 接触矩阵。
- 预处理：设置最大序列长度为 1022，去除过长或无效序列。
- 数据划分：数据集按 7:3 划分为训练集和验证集，固定种子为 42。测试集采用独立的 ArchiveII 数据集进行最终基准评估。
- 测试集说明：测试集不仅包含了训练集里 RNA 的种类，还有其他种类的 RNA，如 23sRNA 等，能够有效评估模型的泛化能力。

二、评估设置

2.1 评估指标选择

本实验选择了准确率、精确率、召回率和 F1 分数作为评估指标。它们的计算公式如下：

$$\begin{aligned} \text{准确率 } Accuracy &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{精确率 } Precision &= \frac{TP}{TP + FP} \\ \text{召回率 } Recall &= \frac{TP}{TP + FN} \\ F1 &= 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \end{aligned} \quad (1)$$

符号说明：

- **TP (True Positive)**: 真实结构中存在碱基配对，且模型正确预测为配对。
- **TN (True Negative)**: 真实结构中无配对，且模型正确预测为无配对。
- **FP (False Positive)**: 真实结构中无配对，但模型错误预测为配对。
- **FN (False Negative)**: 真实结构中存在配对，但模型错误预测为无配对。

采用这些评估指标的原因：

1. 应对数据类别不平衡：

RNA 二级结构预测生成的接触图是一个高度稀疏的矩阵。在 $L \times L$ 的矩阵中，绝大多数位置是未配对的 (0)，只有极少数位置存在碱基配对 (1)。在这种情况下，如果模型倾向于预测全 0，虽然能获得极高的准确率，但完全失去了预测结构的意义。因此，单纯依赖准确率不足以客观评估模型性能。

2. 综合考量查准与查全：

精确率反映了模型预测出的配对有多少是可信的，而召回率反映了模型找出多少真实的配对结构。为了平衡这两者，本实验采用 F1 分数作为核心评价指标。F1 分数是精确率和召回率的调和平均数，只有当两者都较高时，F1 分数才会高，这最能体现模型在稀疏矩阵预测任务中的综合表现。

2.2 测试数据集及划分标准

依据实验要求，测试数据集应完全独立于模型训练与调参过程。本实验采用了 ArchiveII 数据集作为最终的基准测试集。

- 独立性说明：

ArchiveII 数据集中的序列与训练集（RNAStrAlign）虽有部分生物学上的同源性，但在本实验中，ArchiveII 被严格隔离，从未参与训练或验证阶段的任何计算。

- 评估目的：

使用该数据集旨在客观评估模型在未见过的真实 RNA 序列上的预测性能，验证模型的泛化鲁棒性。

三、代码实现

3.1 基于预训练模型的特征提取

本实验的核心创新之一在于摒弃了传统的独热编码（One-hot）方式，转而采用在大规模未标注数据上进行过自监督学习的 **RNA-FM** 模型作为特征提取器。该模型能够捕捉序列内部深层的进化语义信息与上下文依赖关系。

1. 模型加载与初始化

我们在代码中通过调用接口加载预训练权重。为了保证特征提取器的稳定性，我们在初始化阶段冻结了骨干网络的全部参数，仅将其作为静态的特征生成器使用。

2. 第 12 层特征截取

根据相关文献研究，预训练模型的深层输出通常包含更丰富的结构化信息。因此，本实验并未直接使用模型的最终输出，而是提取了第 12 层变换器（Transformer）的隐藏状态作为下游任务的输入。

3. 维度说明

对于长度为 L 的输入序列，经过分词处理后进入模型，提取出的特征矩阵维度为 $L \times 640$ 。其中 640 维的向量代表了每个碱基在进化空间中的高维语义表示。

```
153 with torch.no_grad(): # 确保骨干不计算梯度
154     results = self.backbone(tokens, repr_layers=[12])
```

Figure 1: 特征提取模块代码实现

```
1 # 加载预训练的 RNA-FM 模型
2 # model_location: 预训练权重文件的本地路径
3 self.backbone, self.alphabet = fm.pretrained.rna_fm_t12(
4     model_location=rna_fm_model_path
5 )
6
7 # 提取第 12 层的隐藏状态作为特征, repr_layers=[12] 指定提取层数
8 with torch.no_grad():
9     results = self.backbone(tokens, repr_layers=[12])
10
11 # 获取特征张量, 维度为 [批次大小, 序列长度, 640]
12 embedding = results["representations"][12]
```

3.2 维度变换与成对特征构建

本实验的核心挑战在于：如何将一维的序列特征转化为二维的结构特征？因为 RNA 的二级结构本质上是一个 $L \times L$ 的碱基配对矩阵，而 RNA-FM 输出的是 $L \times D$ 的序列向量。为了解决这一维度跨越问题，本实验设计了外积拼接模块。

该模块利用广播机制，显式地构建了序列中任意两个位置 i 和 j 之间的相互关系。具体实现步骤如下：

- 行特征广播：

将输入特征矩阵 X 在“列”维度上进行复制扩展。对于输出张量的每一个位置 (i, j) ，填充第 i 个碱基的特征向量。这在几何上表现为将 $L \times D$ 的特征板水平拉伸，形成具有“水平条纹”特征的立方体张量。

- 列特征广播：

将输入特征矩阵 X 在“行”维度上进行复制扩展。对于输出张量的每一个位置 (i, j) ，填充第 j 个碱基的特征向量。这在几何上表现为将 $L \times D$ 的特征板垂直拉伸，形成具有“垂直条纹”特征的立方体张量。

- 特征融合：

将上述两个张量在通道维度进行拼接。最终生成的特征体 Z 中的每一个点 (i, j) 都同时包含了第 i 个碱基和第 j 个碱基的特征信息，特征维度翻了一倍。

```
1 # 输入 x 形状: [B, L, D]
2
3 # 1. 扩展维度并复制, 构建"行"特征
4 # x.unsqueeze(2) -> [B, L, 1, D]
5 # .expand -> [B, L, L, D]
6 x_row = x.unsqueeze(2).expand(-1, -1, L, -1)
7
8 # 2. 扩展维度并复制, 构建"列"特征
9 # x.unsqueeze(1) -> [B, 1, L, D]
10 # .expand -> [B, L, L, D]
11 x_col = x.unsqueeze(1).expand(-1, L, -1, -1)
12
13 # 3. 在最后一个维度(特征维)进行拼接
14 # 最终形状: [B, L, L, 2D]
15 out = torch.cat((x_row, x_col), dim=-1)
```

```
9 class OuterConcat(nn.Module):
10     """
11     外部串联层 (Outer Concatenation)
12     功能: 将 1D 序列特征扩展为 2D 配对特征图。
13     逻辑: [B, L, D] -> [B, L, L, 2D]
14     """
15     def __init__(self):
16         super(OuterConcat, self).__init__()
17
18     def forward(self, x):
19         seq_len = x.shape[1]
20         # x_row: 扩展第2维 -> [B, L, L, D]
21         x_row = x.unsqueeze(2).expand(-1, -1, seq_len, -1)
22         # x_col: 扩展第1维 -> [B, L, L, D]
23         x_col = x.unsqueeze(1).expand(-1, seq_len, -1, -1)
24         # 拼接
25         out = torch.cat((x_row, x_col), dim=-1)
26         return out
```

Figure 2: Outer Concatenation 维度变换代码实现

3.3 结构预测与解码

本模块是模型的核心决策层，负责从高维特征中解析出稀疏的配对模式，并输出最终的概率矩阵。整个解码过程分为降维预处理、深度特征提取和概率输出三个阶段。

1. 降维预处理:

为了降低计算复杂度，模型首先引入了一个全连接层作为瓶颈层。由于上一阶段生成的特征体通道数高达 1280 维（2D），直接进行卷积运算开销过大。该层将特征通道数大幅压缩至 128 维（记为 H，为放缩后的维度），在保留关键结构信息的同时显著减少了参数量。随后，通过维度置换操作，将数据调整为适配二维卷积的格式（[B, L, L, H] -> [B, H, L, L]）。

2. 深度特征提取:

模型采用了包含 16 个模块的深度残差网络（Resnet）。与传统的卷积网络不同，本项目采用了“瓶颈式”的三层卷积结构，包含 $1 \rightarrow 3 \rightarrow 1$ 卷积。层间均使用 Relu 函数进行激活。此外，残差连接机制贯穿每个模块，确保了梯度在深层网络中的有效传播，使模型能够捕捉长距离的碱基依赖关系。

3. 概率输出:

最后，经过深层提取的特征被送入输出层。该层通过一个卷积核大小为 3 的二维卷积层，将 128 个特征通道压缩为 1 个通道，得到最终的配对得分矩阵。在推理阶段，该矩阵的数值通过激活函数映射到 0 到 1 区间，表示任意两个碱基形成配对的置信度。

```
67 class ClassifierHead(nn.Module):
68     """
69     预测头（仿照 RNARET 的 ResNet2D_classifier）
70     流程: OuterConcat -> Linear降维 -> ResNet提取 -> Conv输出
71     """
72     def __init__(self, in_dim=640, hidden_dim=128, num_blocks=16, kernel_size=3):
73         super(ClassifierHead, self).__init__()
74
75         # 1. 维度转换组件
76         self.outer_concat = OuterConcat()
77
78         # 2. 降维层（输入是 2*in_dim，因为做了拼接）
79         self.linear_in = nn.Linear(2 * in_dim, hidden_dim)
80
81         # 3. 特征提取器
82         self.resnet = ResNet2D(hidden_dim, num_blocks, kernel_size)
83
84         # 4. 输出层（输出 1 个通道，代表配对得分）
85         self.conv_out = nn.Conv2d(hidden_dim, 1, kernel_size=kernel_size, padding='same')
```

Figure 3: ClassifierHead 的部分代码

```
1 # 1. 降维层：将 1280 维的高维特征压缩为 128 维
2 self.linear_in = nn.Linear(2 * in_dim, hidden_dim)
3 # 2. 特征提取器：堆叠 16 层残差网络
4 self.resnet = ResNet2D(hidden_dim, num_blocks, kernel_size)
5 # 3. 输出层：将特征通道压缩为 1，输出配对得分
6 self.conv_out = nn.Conv2d(hidden_dim, 1, kernel_size=kernel_size,
7                             padding='same')
8
9 # 前向传播流程
10 x = self.outer_concat(x)      # 维度变换
11 x = self.linear_in(x)        # 线性降维
12 x = x.permute(0, 3, 1, 2)    # 调整维度顺序以适配卷积
13 x = self.resnet(x)           # 深度残差提取
14 x = self.conv_out(x)         # 输出最终预测
```

3.4 运行结果截图

由于训练时间很长，平均每个 Epoch 需要约 2 小时。因此我选择设计一段可以从前一个 Epoch 开始训练的代码，也因此我的输出结果并不能展示整个训练过程。但是我将运行结果写入了日志，下面将以日志截图代替训练的输出结果截图。忽略其中的 Acc，因为我一开始计算了 Acc，之后为了节省时间没有计算 Acc 指标，将其令为 0。最终效果可见上传的 train.log 文件，在这里仅展示训练了 1 个 epoch 的模型接着训练的命令截图。

```
(RNAfm) admin@x11ro4z096fcyjt:~/Desktop/My_RNA_Project$ python ssp.py --data_path
~/data/RNAstrAlign --batch_size 2 --accum_steps 4 --max_len 1022 --blocks 16 --epo
chs 10 --save_dir checkpoints/auto_run
2025-12-30 13:25:56,516 - INFO - 启动训练 (AMP+梯度累积) | 设备: cuda
2025-12-30 13:25:56,516 - INFO - 策略: Batch=2 x Accum=4 = 等效 8
2025-12-30 13:25:56,516 - INFO - 1. [Data] 初始化数据流水线...
正在递归扫描目录: /home/admin/data/RNAstrAlign ...
✅ 数据集初始化: 找到 30879 个样本
2025-12-30 13:25:57,577 - INFO - 训练集: 21615 | 验证集: 9264
2025-12-30 13:25:57,577 - INFO - 2. [Model] 构建模型...
正在加载 RNA-FM 权重: ./RNA-FM/pretrained/RNA-FM_pretrained.pth
✅ RNA-FM 骨干参数已冻结 (Frozen)。
2025-12-30 13:25:58,867 - INFO - 3. [Optimizer] 配置优化器...
/home/admin/Desktop/My_RNA_Project/ssp.py:164: FutureWarning: `torch.cuda.amp.Grad
Scaler(args...)` is deprecated. Please use `torch.amp.GradScaler('cuda', args...)`
instead.
  scaler = torch.cuda.amp.GradScaler()
2025-12-30 13:25:58,868 - INFO - 发现存档, 正在恢复: checkpoints/auto_run/last_
checkpoint.pth
2025-12-30 13:25:59,505 - INFO - ✅ 成功恢复! 将从 Epoch 2 继续训练 (当前最佳 F1:
0.0000)
2025-12-30 13:25:59,505 - INFO -
=== Epoch 2/10 ===
Training: 0% | 4/10808 [00:02<1:50:34, 1.63it/s, loss=0.000694]
```

Figure 4: 训练 1 个 epoch 后接着训练的截图

四、实验结果

4.1 模型参数及结构描述

本实验提出的 RNA 二级结构预测模型采用了“预训练语言模型 + 深度残差网络”的混合架构。整体结构如图 5 所示，主要包含以下三个核心处理阶段：

1. 序列特征嵌入 (Embedding Stage):

利用预训练的 **RNA-FM** 模型作为特征提取器。对于长度为 L 的输入 RNA 序列，RNA-FM 输出包含 RNA 序列信息的嵌入向量，其维度为 $L \times D$ 。为节省计算资源并防止过拟合，本实验冻结了 RNA-FM 的骨干参数。

2. 成对特征构建 (Pairwise Representation Construction):

为了将一维序列信息转化为矩阵，模型引入了外部拼接机制。如图 5 所示，模型分别在“行”维度和“列”维度对特征进行广播 (Broadcasting)。随后将两者在通道维度进行拼接，生成形状为 $[L, L, 2D]$ 的二维特征张量。这一步使得每个空间位置 (i, j) 都包含了第 i 个碱基和第 j 个碱基的组合特征。

3. 深度特征提取与预测 (Deep Feature Extraction):

生成的二维特征图被送入包含 16 个残差块 (Residual Blocks) 的 **ResNet** 中。利用卷积层和残差连接，模型能够有效捕捉长距离的碱基依赖关系。最终，通过一个 1×1 的卷积层将通道压缩，并经过 Sigmoid 激活函数输出大小为 $L \times L$ 的概率矩阵，表示碱基配对的置信度。

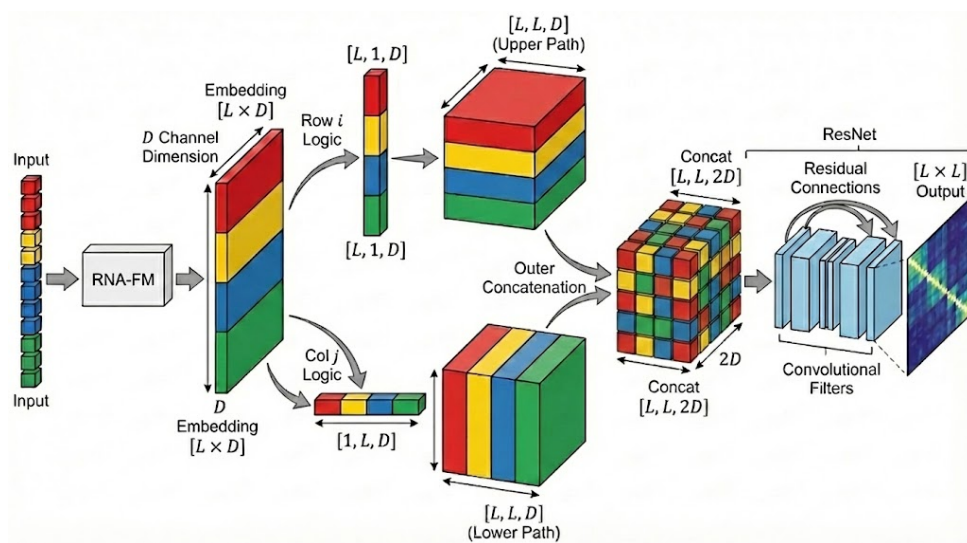


Figure 5: 基于 RNA-FM 与 ResNet 的 RNA 二级结构预测模型架构示意图。

4.2 训练参数

本实验的具体训练超参数设置如表 1 所示。为了解决正负样本（配对与非配对）极度不平衡的问题，损失函数中引入了正样本权重（pos_weight）。

Table 1: 实验模型参数与训练超参数设置

参数名称 (Parameter)	数值 (Value)
预训练模型 (Pre-trained Model)	RNA-FM (冻结参数)
放缩后的维度 (Hidden, H)	128
ResNet 层数 (Blocks)	16
最大序列长度 (Max Length, L)	1022
优化器 (Optimizer)	AdamW
学习率 (Learning Rate)	1×10^{-4}
批大小 (Batch Size)	2
梯度累积步数 (Accumulation Steps)	4
损失函数 (Loss Function)	二元交叉熵损失
正样本权重 (pos_weight)	10
训练轮数 (Epochs)	10

4.3 评估指标

4.3.1 训练与验证: RNAStrAlign

为了评估模型在训练过程中的收敛情况及泛化能力，我们绘制了训练集与验证集的损失变化曲线（图 6）以及验证集上的关键性能指标变化曲线（图 7）。

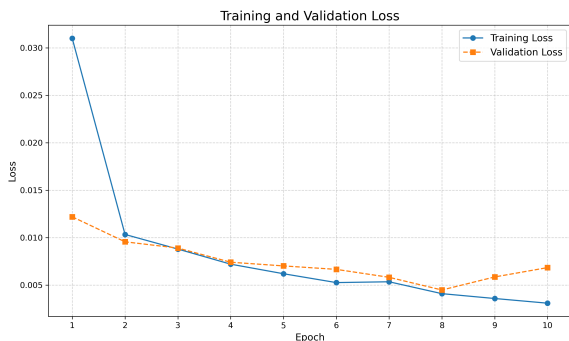


Figure 6: 训练与验证损失收敛曲线

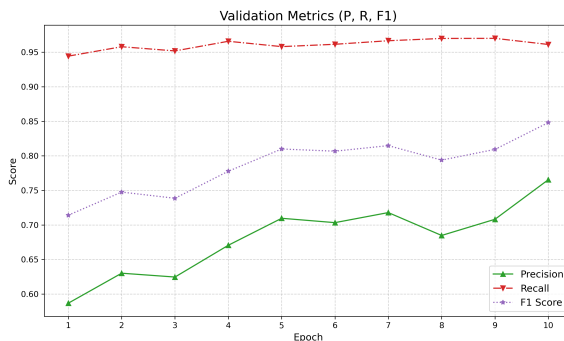


Figure 7: 验证集中 P, R 与 F1 变化曲线

结果分析：

• 损失收敛性：

如图 6 所示，Epoch 8 时，训练损失与验证损失均呈现下降趋势。但是之后验证损失呈现上升趋势，说明模型在训练过程中出现了过拟合现象，不过并不严重。

• 指标变化：

如图 7 所示，F1 分数随训练过程稳步提升。模型在保持较高召回率的同时，精确率逐步改善，表明模型区分真假阳性配对的能力在不断增强。

4.3.2 基准测试：ArchiveII

在模型训练完成后，我们在独立的测试集（Benchmark Dataset）上进行了最终性能评估。该测试集的数据分布与训练集隔离，能够客观反映模型的泛化性能。最终的评估结果如表 2 所示。

Table 2: 测试集最终性能评估结果

数据集	准确率	精确率	召回率	F1	AUC
ArchiveII	0.9978	0.7148	0.7432	0.7287	0.9632

结果分析：

1. 极高的准确率与稀疏性：

实验结果显示，模型的准确率高达 **99.78%**。虽然很高，但是 RNA 二级结构接触图具有极高的稀疏性，因为绝大多数碱基不发生配对，即负样本占绝大多数。因此，准确率主要反映了模型对负样本（TN）的正确排除能力，而非单一的核心性能指标。

2. 综合性能：

F1 分数为 **0.7287**，该指标表明模型在正样本预测上取得了良好的平衡，既能以较高的比例覆盖真实配对，又能将误报率控制在合理范围内。

3. AUC：

AUC 达到了 **0.9632**，这表明模型输出的预测概率具有极高的可信度。在不同的分类阈值下，模型能够极其有效地将真实配对与非配对区分开来，证明了模型架构在特征提取方面的有效性。

4.4 ROC 曲线

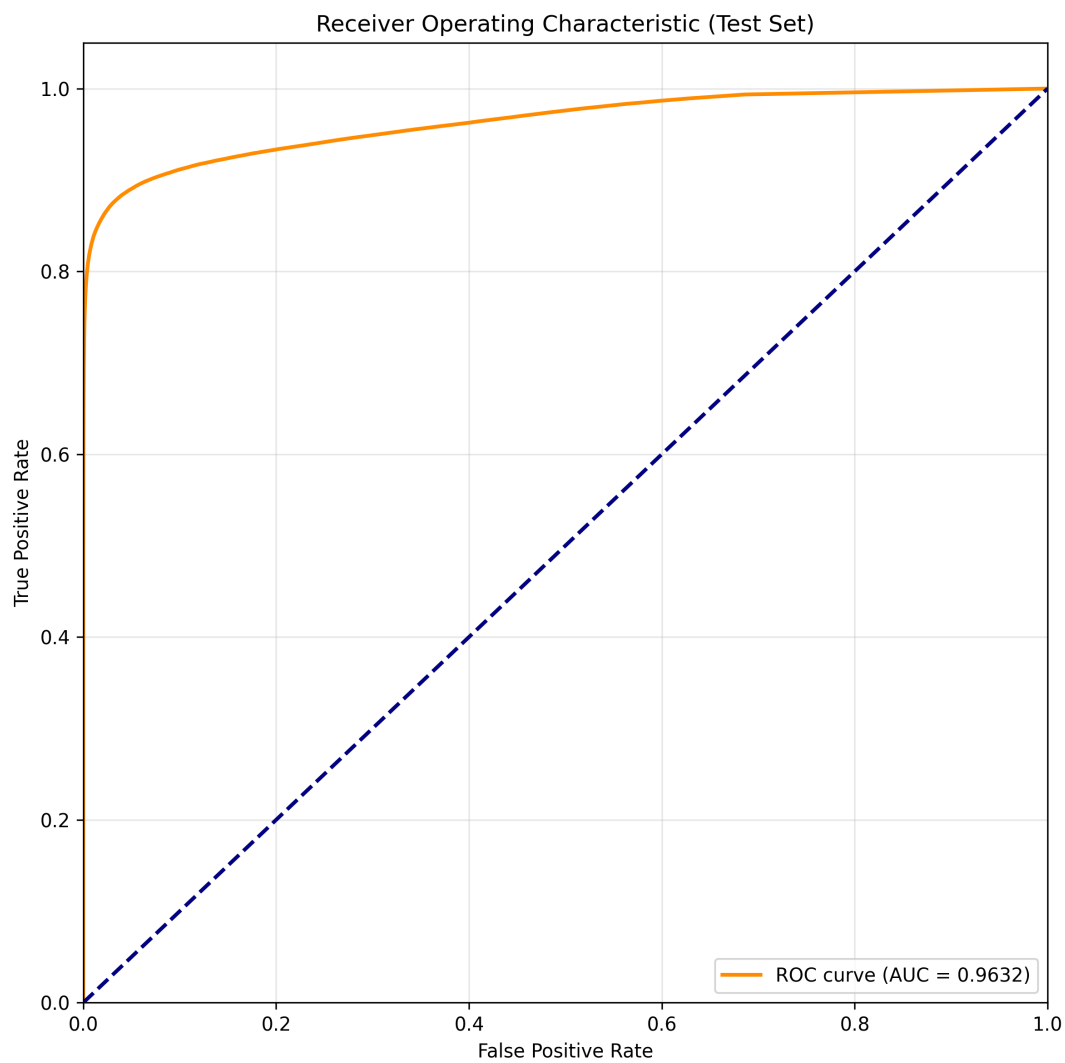


Figure 8: 模型在测试集上的 ROC 曲线

五、前端界面展示

5.1 首页

首页页面过长，为了报告的美观，仅展示部分，具体见 README.md

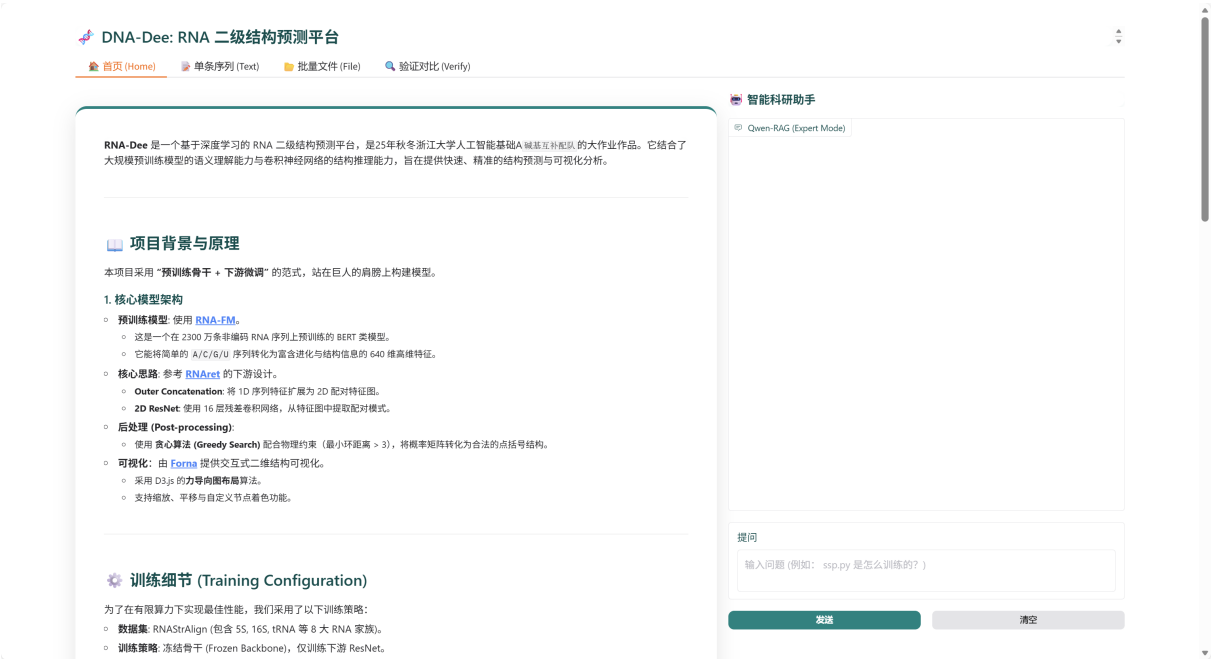


Figure 9: 首页预览图

同时，可以和右侧的阿里云 AI 对话，该 AI 通过检索增强技术（RAG），获得了所有的代码文件，可以回答大多数问题：



Figure 10: AI 对话界面展示 (1)



Figure 11: AI 对话界面展示 (2)

5.2 单条序列

在未填入序列的情况下：

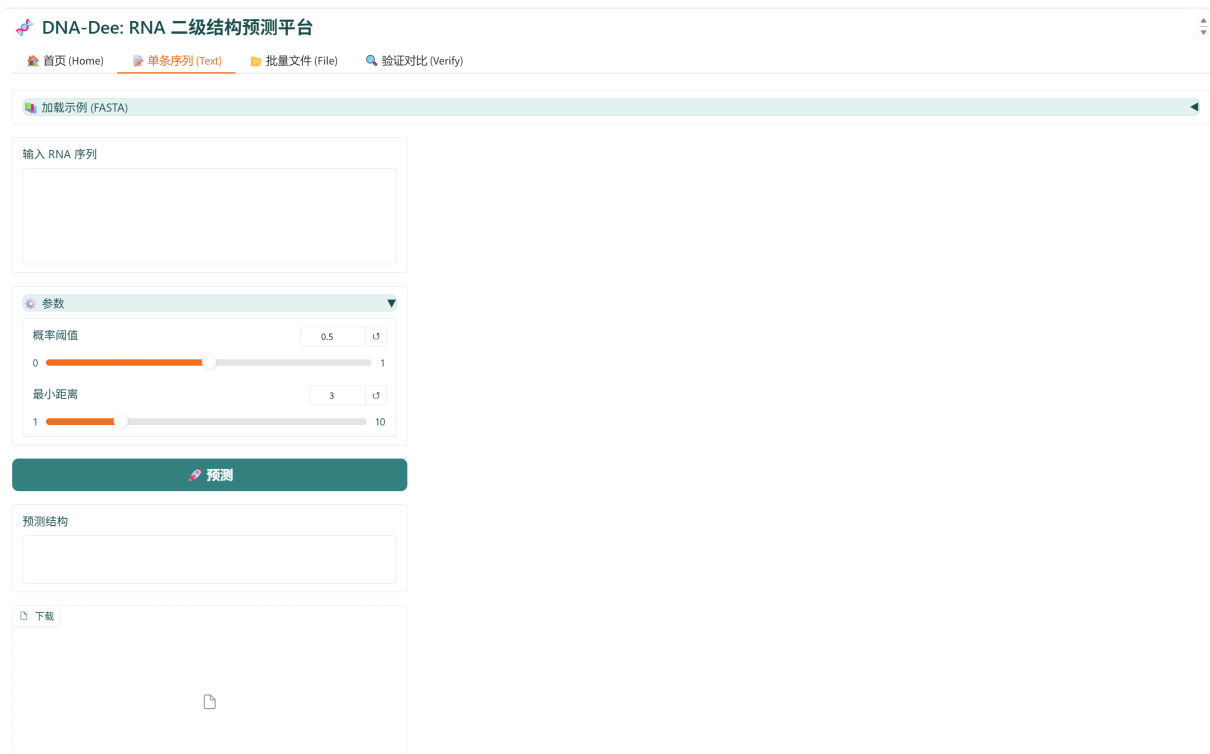


Figure 12: 单条序列-未填入

填入序列后的预测结果：

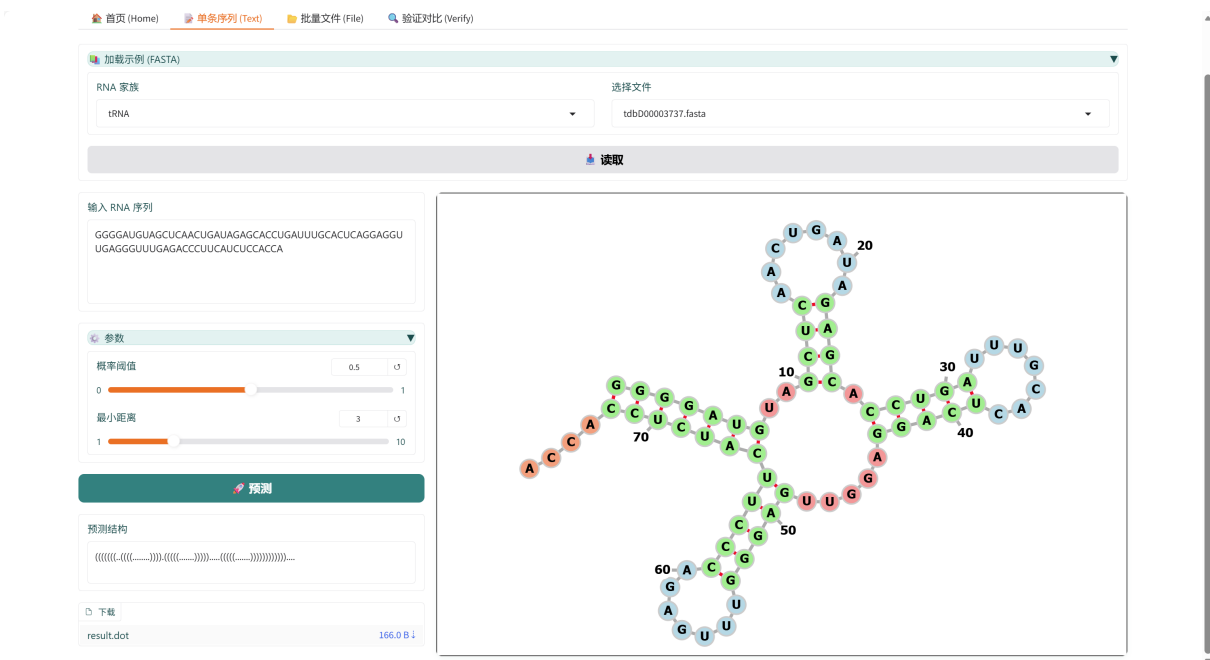


Figure 13: 单条序列-已填入

若目前没有可用的 fasta 序列，可以选择训练集里 RNA 家族中的某一个示例，如图所示：

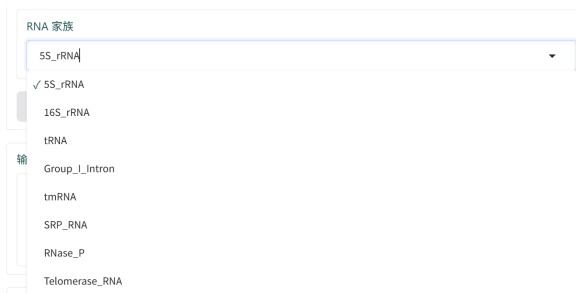


Figure 14: 单条序列-选择 RNA 家族

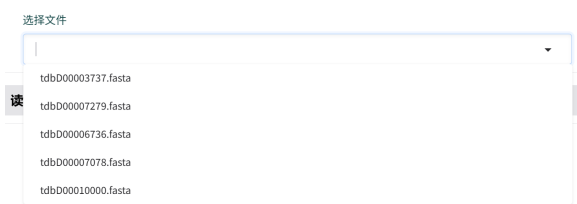


Figure 15: 单条序列-选择 fasta

5.3 批量预测

在批量预测页面，可以上传多个 fasta 文件进行预测：



Figure 16: 批量预测-未填入

上传文件后的界面：

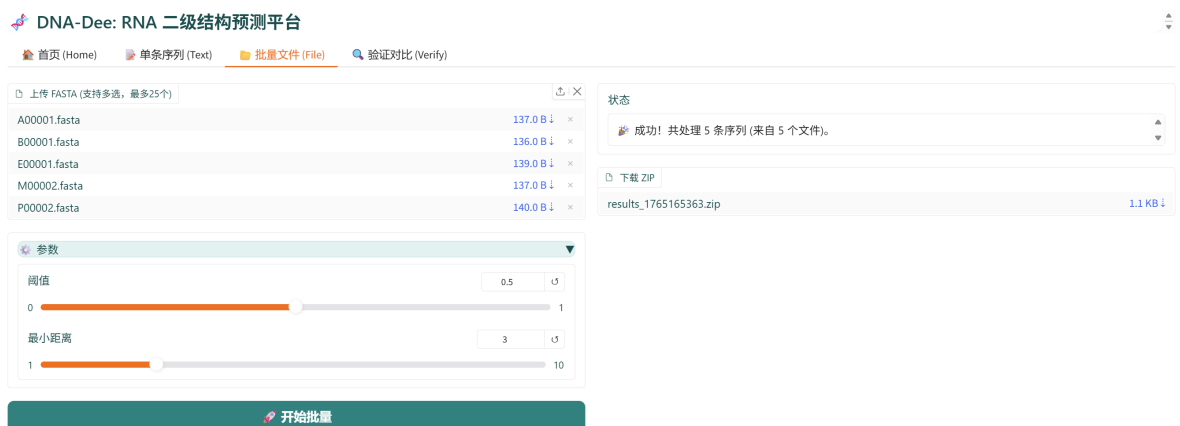


Figure 17: 批量预测-已填入

5.4 验证对比

在该页面，可以选择带有真实结构的 bpseq 文件进行对比验证：

The screenshot shows the 'DNA-Dee: RNA 二级结构预测平台' (DNA-Dee: RNA Secondary Structure Prediction Platform) interface. The top navigation bar includes links for '首页 (Home)', '单条序列 (Text)', '批量文件 (File)', and '验证对比 (Verify)'. The 'Verify' section is active, showing a '加载测试集示例 (.bpseq)' dropdown menu. Below this, there are two dropdown menus: 'RNA 家族' (RNA Family) set to '5S_rRNA' and '选择文件' (Select File) set to 'B02402.bpseq'. A '载入' (Load) button is present. Below the dropdowns, there is a section for '上传 bpseq 文件 (含真实结构)' (Upload bpseq file (with real structure)) with a file upload icon and text '将文件拖放到此处 - 或 - 点击上传' (Drag file here - or - click upload). To the right, there is a '结构预测图' (Structure Prediction Diagram) section. Below these, there is an '实时调参' (Real-time Parameter Adjustment) section with sliders for '概率阈值' (Probability Threshold) set to 0.5 and '最小距离' (Minimum Distance) set to 3. At the bottom, there is a '开始对比' (Start Comparison) button.

Figure 18: 验证对比-未填入

同样，可以选择不同 RNA 家族的不同示例进行对比，包含训练集和测试集：

This screenshot shows the 'RNA 家族' (RNA Family) dropdown menu. The selected option is '5S_rRNA'. The list of options includes: '5S_rRNA', '16S_rRNA', 'tRNA', 'Group_I_Intron', 'tmRNA', 'SRP_RNA', 'RNase_P', and 'Telomerase_RNA'.

Figure 19: 验证对比-选择 RNA 家族

This screenshot shows the '选择文件' (Select File) dropdown menu. The selected option is 'B02402.bpseq'. The list of options includes: 'B02402.bpseq', '(测试集)_5s_Nephila-clavipes-1.bpseq', '(测试集)_5s_Methanobrevibacter-ruminantium-1.bpseq', '(测试集)_5s_Christiansenia-pallida-1.bpseq', 'B05885.bpseq', 'B01737.bpseq', 'B03369.bpseq', and 'B00757.bpseq'.

Figure 20: 验证对比-选择 bpseq

六、经验及总结

6.1 分点总结

除了第一条外，我将简单列举我遇到的问题与解决方法：

1. 最严重、代价最大

- 所遇问题：我在第一次训练时，当时还没出大作业模板，由于我的训练意识比较薄弱，没有考虑到使用固定种子去划分验证集和训练集，导致我最后发现我无法找到验证集去计算指标。
- 解决方法：我重新设计了增加计算准确率的函数，并且稍微改了下训练代码 `ssp.py`，然后使用了全局种子 42，最终得以解决该问题。
- 自我总结：虽然我最后重新解决了该问题，计算出了每一个 Epoch 的四个指标，但是浪费了我的阿里云 AI 工作站的算力。最后我训练了 10 个 Epoch，消耗了我的大量算力，本来我还打算留不少算力用于训练《医工交叉融合的人工智能》里的大作业，现在还得找别人的账号或者充钱了。

2. 追求完美

- 所遇问题：在第二次训练的时候，由于为了追求获得更多的指标，我决定修改计算指标的 `metrics` 函数，在每次 Epoch 跑完后都对验证集同时计算准确率、精确率、召回率和 F1 分数。但是，由于计算准确率太复杂，导致在验证集的评估上会花费大量时间，浪费了我的算力。
- 解决方法：考虑到目前有限的算力，我不得不放弃计算过程中对验证集的准确率。
- 自我总结：有的时候，追求完美并不是一件好事，尤其是在算力有限的情况下，更何况在样本种类失衡的状况下，准确率显然不如另外三个指标重要。我以后在训练模型时，会优先考虑算力的消耗，避免不必要的计算。

3. 数据加载

- 所遇问题：我在加载数据时，会报错文件不存在，最后发现该函数只能进入一个文件夹，不能进入子文件夹并检索数据。
- 解决方法：我修改了加载数据的函数，使用 `os.walk` 替代 `glob`，`os.walk` 会自动进入所有子文件夹，解决 RNAStrAlign 的嵌套问题。

- 自我总结：以后在加载数据时，我会检查数据文件夹的结构，如果存在子文件夹，我会优先考虑使用 `os.walk` 进行递归检索。

4. 搬运代码

- 所遇问题：由于我本地的电脑显存仅有 6GB，而训练该模型需要很高的算力，因此我决定租一个阿里云的 AI 工作站，在上面进行训练。可是后面我发现，重新配置环境极其麻烦，因为其网络有限，很多包无法下载，而只要使用就会消耗算力时间。同时，我的数据过大，搬运到工作站上也很麻烦。
- 解决方法：数据方面，我向客服发出工单，客服告诉我如何高速传输数据；同时，我将环境压缩成压缩包，在工作站里访问本地的文件夹，将本地的环境压缩包在工作站里解压，这样就可以不依赖于网络就快速配置好环境了。
- 自我总结：幸好 AI 工作站可以直接访问我本地的文件夹，否则我可能需要花费非常非常多的时间去配置环境，以至于放弃使用 AI 工作站。

5. 显存爆炸

- 所遇问题：在训练过程中，我使用的是阿里云 48G 显存的 GPU，但是在训练的时候，要么就是显存不够，报错 `CUDA out of memory`，要么就是显存利用率少，上监控一看我的显存利用率才 30%，浪费我租工作站的钱。
- 解决方法：经过一晚上的调试，我终于找到一个合适的超参数，使得显存不会爆掉，同时显存利用率也能达到 85% 以上。我将 `batch_size` 调小为 2，同时将模型的残差网络层数调小为 16 层，最终达到了一个折中的效果。
- 自我总结：在训练过程中，我使用 GPU 训练时，将 `batch_size` 调小，使得显存不会爆掉。

6. RAG 实现

- 所遇问题：在搭建 WebUI 的“智能问答助手”模块时，我遭遇了 Gradio 版本更新带来的巨大阵痛。原本参考教程编写的 `rag_chat` 函数在运行时疯狂报错。查阅文档后才发现，新版 Gradio 的 Chatbot 组件已经弃用了元组格式，强制要求使用字典列表格式。
- 解决方法：我不得不重写了整个问答逻辑，适配了最新的 `messages` 格式。此外，我编写了一个后端函数，在启动时自动递归扫描我指定的文件并注入到系统提示词中，实现了 RAG。

- 自我总结：开源社区更新太快了，教程往往是滞后的。在做工程实现时，不能死板照抄代码，必须具备阅读最新官方文档的能力。

7. 追求美观

- 所遇问题：为了追求美观，我用 Nano banana Pro 绘制了背景图，很有 RNA 结构的生物科技感，并希望按钮颜色能吸取背景中的主色调，但在实际操作中发现，我实在找不到如何更改可视化界面的背景图，且按钮的默认色调极难通过简单参数覆盖。
- 解决方法：我利用取色器提取了背景图中的关键色值，不再依赖默认主题，而是通过 CSS 注入的方式，强制重写了 `.gradio-container` 的背景属性和 `button.primary` 的颜色样式。经过多次尝试，我终于更改了按钮的颜色，更换了整体的主题。
- 自我总结：虽然为了调这几个颜色花费了很多时间，甚至背景图最终还是没有整出来，但我在这其中学到的不仅是如何更换主题颜色，还学到了如何上网搜索教程以满足自己的美观需求。

8. 急于求成

- 所遇问题：我的模型在验证集上跑的时候，用时非常长，足足 40 分钟。于是我尝试修改其 `batch_size`，从 2 增加到 16。结果吃完饭回来，发现显存已经爆了，我白白浪费了一个半小时的算力。之后，我将 `batch_size` 改成 4，发现显存占用从 85% 降低到了约 60%，反而降低了速度。
- 解决方法：我将 `batch_size` 改回来了。
- 自我总结：我太激进了，同时太冲动了。我忘记了我的保存模型代码是在验证评估之后，因此一旦显存爆了，那我的模型就没了。同时，我也只是想当然的改了 `batch_size`，结果反而又更加耗时了。这一连串操作一共让我白白浪费了快 2 个小时，都够我训练一个 Epoch+ 验证了。我的心急反而让我付出了更多的代价，实在糊涂！不过说实话，我到现在都不知道为什么我的 `batch_size` 变大反而显存占用更少了，导致速度更慢了。等放假我要好好查明原因，避免下次再犯错。

9. 自动运行

- 所遇问题：我刚开始尝试运行的时候，总是在不知不觉间爆显存，浪费了算力。

- 解决方法：我写了一个 `run_with_restart.py`，可以在显存爆了后自动重新运行。
- 自我总结：虽然感觉没能解决根本问题，但至少获得了一些经验，而且之后也没有爆过显存。

6.2 个人总结

在该大作业中，我实现了第一次独立开发一个 AI 小项目，并且与我的专业高度相关。通过该实验，我亲自完成了从思路构建、代码实现、模型训练到设计 UI 的完整流程，收获颇丰。为了锦上添花，我决定去获取阿里云的免费限额 API，通过 RAG 的技术接入到我的 UI 界面中，这样在展示的时候别人也可以通过该 AI 助手，快速简单了解我的项目。

由于至始至终都是一个人，我不得不在冬学期初就开始筹划如何开展。刚开始时，是我认识的师兄为我提供了灵感，他之前做过类似的项目，并且给我推荐了 RNA-FM 这个预训练模型，让我设计新的概率预测算法。随后，我花了一周时间去熟悉该模型的使用方法，并且阅读了相关论文，了解其原理。接下来，我开始设计我的模型结构，并且编写代码实现。在这个过程中，我遇到了很多困难，比如如何进行维度变换，如何设计残差网络等。通过查阅资料和反复调试，我最终解决了这些问题。

同时，也很感谢朱老师。因为一开始我在学期初就和别人组好队了，但是朱老师很看好我，认为我可以自己一个人做，因此我才下定决心，离开队伍后自行开发。同时，朱老师还为我提供了一些科研经费，让我不那么算力焦虑。不过，至少目前我还可以白嫖阿里云的算力，也很感谢阿里云的算力支持，让我有能力训练模型。我计算了一下，我一共耗时大概 3200 核时，折合人民币大约要 100 元。如果没有阿里云大方送我们算力的话，我是肯定舍不得这么多钱投入到我的作业当中的。